

ShopVerse

Infrastructure Services

Code Touch Points • UI → DB Flows • Logs • Inspection

Traces every request from the React UI through Spring Security, REST Controllers, Use Cases, Repositories and into each of the 8 backing stores. Includes actual class names, method signatures, and exact Docker debug commands.

PostgreSQL

Redis

Cassandra

MongoDB

Neo4j

Elasticsearch

Zookeeper

Kafka

Flow Layer Colour Legend

UI / Frontend	React pages, axios API client, Context
Security	JwtAuthenticationFilter, SecurityConfig
Controller	@RestController, @Controller, @QueryMapping
Service / Use Case	@Service, Use Cases, Saga, Cache Service
Repository / Infra	JPA, Mongo, Cassandra, Neo4j, ES repositories
Database	Actual table / collection / index / topic

PostgreSQL — Primary transactional store — customers, products, orders

Use Case 1: List Products (GET /api/products)

User opens the Products page. UI calls the REST API; Spring Security checks the JWT; ProductController delegates to SearchProductsUseCase which queries PostgreSQL via JPA.

Request Flow (UI → DB)



Key Code Details

ProductController.java (shopverse-web)

```
@RestController @RequestMapping("/api/products")

public class ProductController {

    @GetMapping

    public ResponseEntity>> list(

        @RequestParam(required=false) String keyword,

        @RequestParam(required=false) String category,

        @RequestParam(defaultValue="0") int page,

        @RequestParam(defaultValue="20") int size) {

        return ok(useCase.execute(new SearchProductsQuery(keyword, category, page, size)));

    }

}
```

JpaProductRepository.java (shopverse-infrastructure)

```

public interface JpaProductRepository extends JpaRepository {

    @Query("SELECT p FROM ProductEntity p WHERE LOWER(p.name)"
+ " LIKE LOWER(CONCAT('%', :q, '%')) AND p.active = true")

    List searchByName(@Param("q") String query);

    List findByCategoryAndActiveTrue(String category);

}

```

Use Case 2: Place Order (POST /api/orders)

Authenticated user submits cart. PlaceOrderUseCase coordinates customer load, stock decrement, order creation, loyalty points — all in one @Transactional boundary.

Request Flow (UI → DB)



Key Code Details

PlaceOrderUseCase.java (shopverse-application)

```

@Service @Transactional

public class PlaceOrderUseCase {

    public Order execute(PlaceOrderCommand cmd) {

        Customer customer = customerRepo.findById(cmd.customerId());

        Order order = new Order(customer.getId(), cmd.address());

        for (OrderItemCommand item : cmd.items()) {

            Product p = productRepo.findById(item.productId());

```

```

p.reduceStock(item.quantity()); // domain validation

productRepo.save(p);

order.addItem(new OrderItem(p, item.quantity()));
}

order.confirm();

Order saved = orderRepo.save(order);

eventPublisher.publish(new OrderPlacedEvent(saved));

customer.addLoyaltyPoints(calculatePoints(saved));

customerRepo.save(customer);

return saved;
}
}

```

Getting Logs

Command	What it shows
<code>docker logs shopverse-postgres --tail 100 -f</code>	Follow live container logs
<code>docker logs shopverse-postgres 2>&1 grep ERROR</code>	Filter errors
In <code>application.yml</code> : <code>org.hibernate.SQL: DEBUG</code>	Print every SQL query
In <code>application.yml</code> : <code>org.hibernate.type.descriptor.sql: TRACE</code>	Print bind param values

Inspecting Values

Command	What it shows
<code>docker exec -it shopverse-postgres psql -U shopverse</code>	Open psql shell
<code>\c shopverse then \dt</code>	Connect and list tables
<code>SELECT * FROM orders ORDER BY created_at DESC LIMIT 5;</code>	Latest 5 orders
<code>SELECT * FROM products WHERE active=true LIMIT 10;</code>	Active products
<code>SELECT * FROM flyway_schema_history;</code>	Migration history
<code>EXPLAIN ANALYZE SELECT * FROM products WHERE name ILIKE '%phone%';</code>	Query plan

Pro Tips

- ✓ Use `@Version` on entities for optimistic locking — `PlaceOrderUseCase` relies on it for concurrent orders
- ✓ `application.yml`: `spring.jpa.hibernate.ddl-auto=none` means Flyway owns the schema, not Hibernate
- ✓ `JpaProductRepository.decrementStock()` uses a bulk `UPDATE` — avoids loading entity into memory

Redis — Cache, Pub/Sub, distributed locks, idempotency keys

Use Case 1: Product Cache (GET /api/products/{id})

CachedProductService wraps ProductRepository with @Cacheable. First call hits PostgreSQL and stores result in Redis. Subsequent calls return from Redis without hitting the DB.

Request Flow (UI → DB)



Key Code Details

CachedProductService.java (shopverse-application)

```
@Service
public class CachedProductService {
    @Cacheable(cacheNames="products", key="#id")

    public Product findById(Long id) {
        return productRepo.findById(id) // only runs on cache MISS
        .orElseThrow(() -> new DomainException("P-001", "Not found"));
    }

    @CachePut(cacheNames="products", key="#result.id")
    public Product save(Product product) { return productRepo.save(product); }

    @Caching(evict = { @CacheEvict(cacheNames="products", key="#id"),
        @CacheEvict(cacheNames="products", key="'all'") })
    public void evict(Long id) {}
}
```

Use Case 2: Order Pub/Sub Event (after order placed)

After PlaceOrderUseCase saves an order, OrderKafkaProducer publishes an event, but RedisPubSubPublisher ALSO fires on the 'order-events' channel for real-time UI dashboards.

Request Flow (UI → DB)



Key Code Details

RedisPubSubPublisher.java (shopverse-infrastructure)

```
@Component

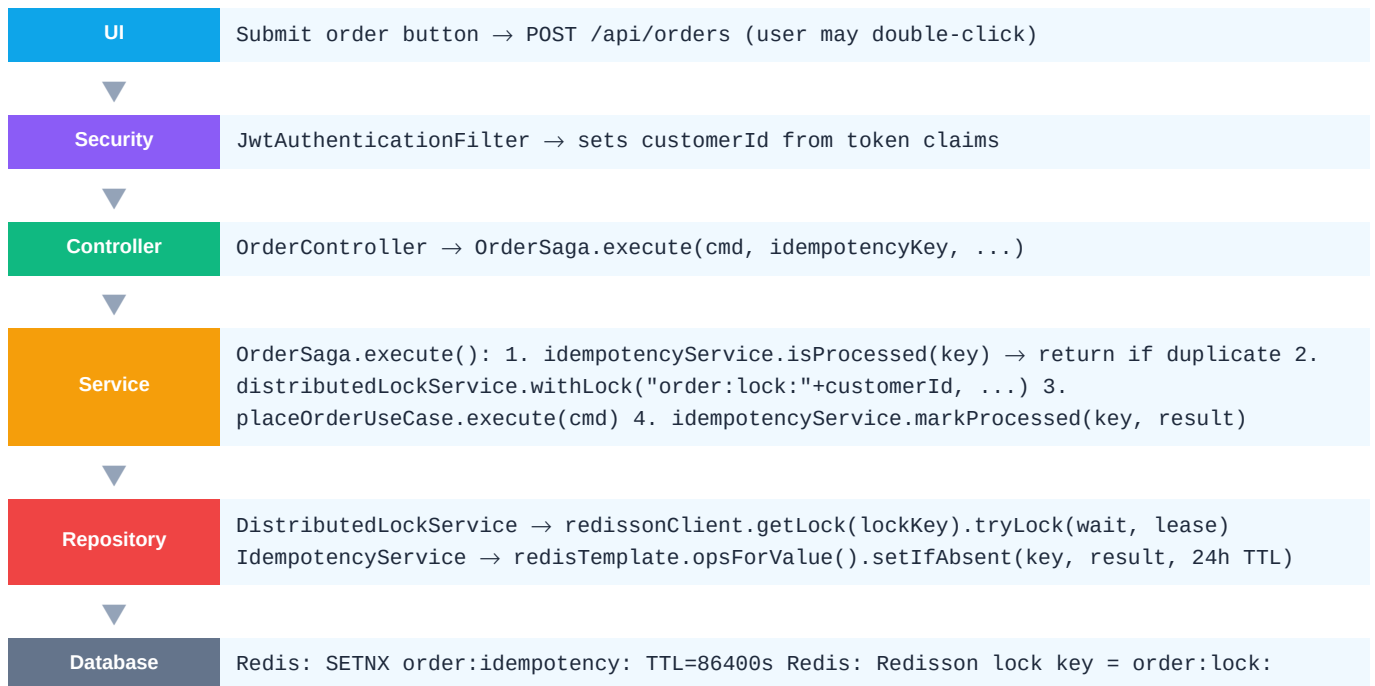
public class RedisPubSubPublisher {
    private final RedisTemplate redisTemplate;
    private final ObjectMapper objectMapper;

    public void publish(String channel, Object message) {
        String payload = objectMapper.writeValueAsString(message);
        redisTemplate.convertAndSend(channel, payload);
    }
}
```

Use Case 3: Distributed Lock in OrderSaga

OrderSaga acquires a Redisson RLock on the customer ID before placing an order, preventing duplicate concurrent orders for the same customer.

Request Flow (UI → DB)



Key Code Details

OrderSaga.java (shopverse-application)

```
@Service
public class OrderSaga {
    public Order execute(PlaceOrderCommand cmd, String idempotencyKey, ...) {
        if (idempotencyService.isProcessed(idempotencyKey))
            return idempotencyService.getResult(idempotencyKey);

        return distributedLockService.withLock(
            "order:lock:" + cmd.customerId(), 5, 30, () -> {
                Order order = placeOrderUseCase.execute(cmd);
                idempotencyService.markProcessed(idempotencyKey, order);
                return order;
            });
    }
}
```

Getting Logs

Command	What it shows
<code>docker logs shopverse-redis --tail 50 -f</code>	Follow live Redis logs
<code>docker exec shopverse-redis redis-cli MONITOR</code>	Stream ALL commands (dev only)

<code>docker exec shopverse-redis redis-cli SLOWLOG GET 10</code>	Last 10 slow commands
<code>docker exec shopverse-redis redis-cli INFO stats</code>	Keyspace hits/misses, ops/sec

Inspecting Values

Command	What it shows
<code>docker exec -it shopverse-redis redis-cli</code>	Open Redis CLI
<code>KEYS products:*</code>	All product cache keys
<code>GET products::1</code>	Cached product with id=1
<code>TTL products::1</code>	Remaining TTL in seconds
<code>SUBSCRIBE order-events</code>	Watch Pub/Sub channel live
<code>KEYS order:idempotency:*</code>	All idempotency keys

Pro Tips

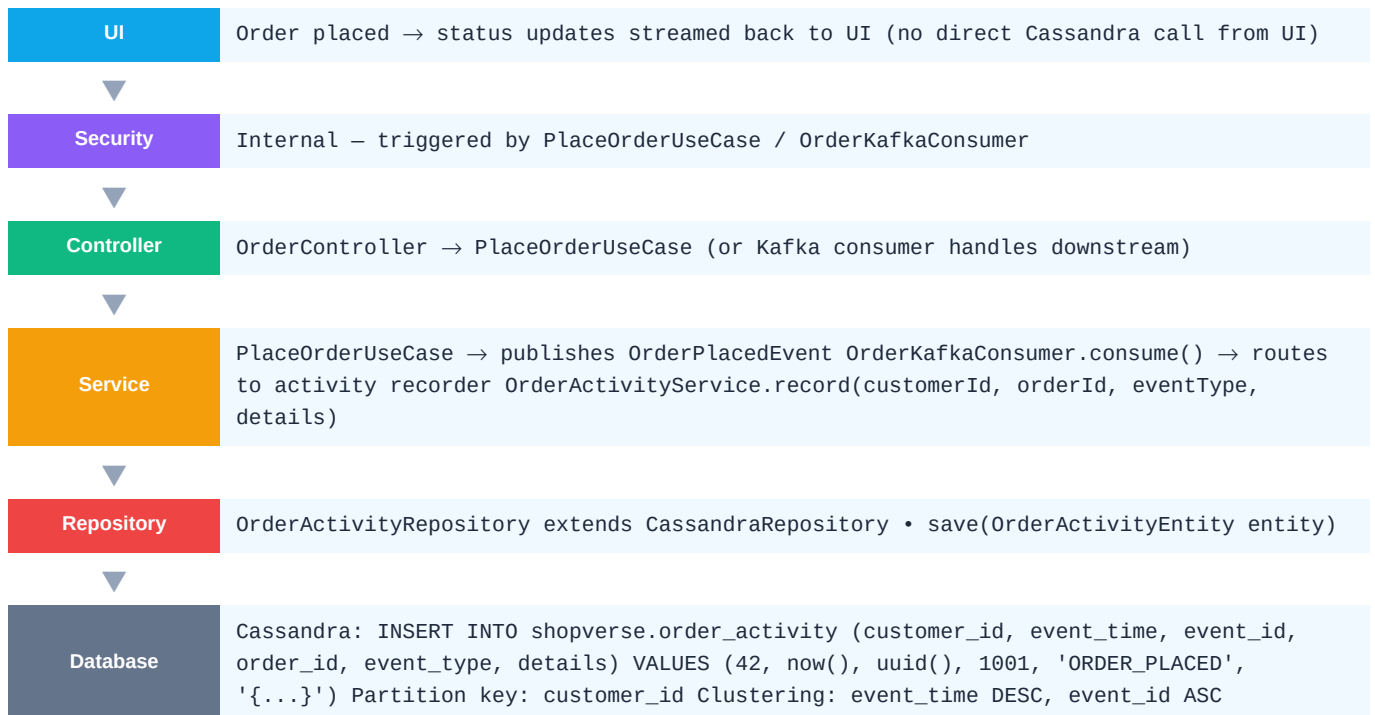
- ✓ MONITOR shows every Redis command in real time — use to verify @Cacheable is hitting cache
- ✓ `spring.cache.redis.time-to-live` in `application.yml` controls default TTL for @Cacheable
- ✓ `RedissonConfig.java` sets password only when `REDIS_PASS` env var is non-blank — fixes empty AUTH bug

Cassandra — Wide-column store — order activity time-series log

Use Case 1: Record Order Activity Event

Every order state transition (placed, confirmed, shipped) writes an immutable event row to Cassandra, partitioned by customer_id for efficient per-customer queries.

Request Flow (UI → DB)



Key Code Details

OrderActivityRepository.java (shopverse-infrastructure)

```
public interface OrderActivityRepository
extends CassandraRepository {

    List
    findByIdOrderByEventTimeDesc(Long customerId);

    @Query("SELECT * FROM order_activity"
    + " WHERE customer_id = ?0 AND event_time > ?1 LIMIT 50")
    List findRecentActivity(Long customerId, Instant since);
}
```

OrderActivityEntity.java (shopverse-infrastructure)

```
@Table("order_activity")
public class OrderActivityEntity {

    @PrimaryKeyColumn(ordinal=0, type=PARTITIONED)
    private Long customerId;

    @PrimaryKeyColumn(ordinal=1, type=CLUSTERED, ordering=DESCENDING)
```

```

private Instant eventTime;

@PrimaryKeyColumn(ordinal=2, type=CLUSTERED)
private UUID eventId;

private Long orderId;

private String eventType;

private String details;

}

```

CassandraConfig.java — bootstrap on startup (shopverse-infrastructure)

```

@Bean(destroyMethod="close") @Primary
public CqlSession cassandraSession(CqlSessionBuilder builder) {
    // Step 1: no-keyspace session to create keyspace
    try (CqlSession boot = CqlSession.builder()
        .addContactPoint(new InetSocketAddress(host, port))
        .withLocalDatacenter(localDc).build()) {
        boot.execute("CREATE KEYSPACE IF NOT EXISTS shopverse ...");
        boot.execute("CREATE TABLE IF NOT EXISTS shopverse.order_activity ...");
    }

    // Step 2: real session with keyspace (used by Spring Data)
    return builder.build(); // builder already has keyspace-name=shopverse
}

```

Getting Logs

Command	What it shows
<code>docker logs shopverse-cassandra --tail 100 -f</code>	Follow live Cassandra logs
<code>docker exec shopverse-cassandra nodetool status</code>	Ring status — all nodes UN=healthy
<code>docker exec shopverse-cassandra nodetool tpstats</code>	Thread pool queue depths
<code>docker exec shopverse-cassandra tail -f /var/log/cassandra/system.log</code>	System log file

Inspecting Values

Command	What it shows
<code>docker exec -it shopverse-cassandra cqlsh</code>	Open CQL shell
<code>USE shopverse; DESCRIBE TABLE order_activity;</code>	Schema with partition/clustering keys
<code>SELECT * FROM order_activity WHERE customer_id=1 LIMIT 10;</code>	Activity for customer 1
<code>SELECT * FROM order_activity WHERE customer_id=1 AND event_time > '2024-01-01' LIMIT 10;</code>	Time-range query
<code>docker exec shopverse-cassandra nodetool tablestats shopverse.order_activity</code>	Read/write latency, row count

Pro Tips

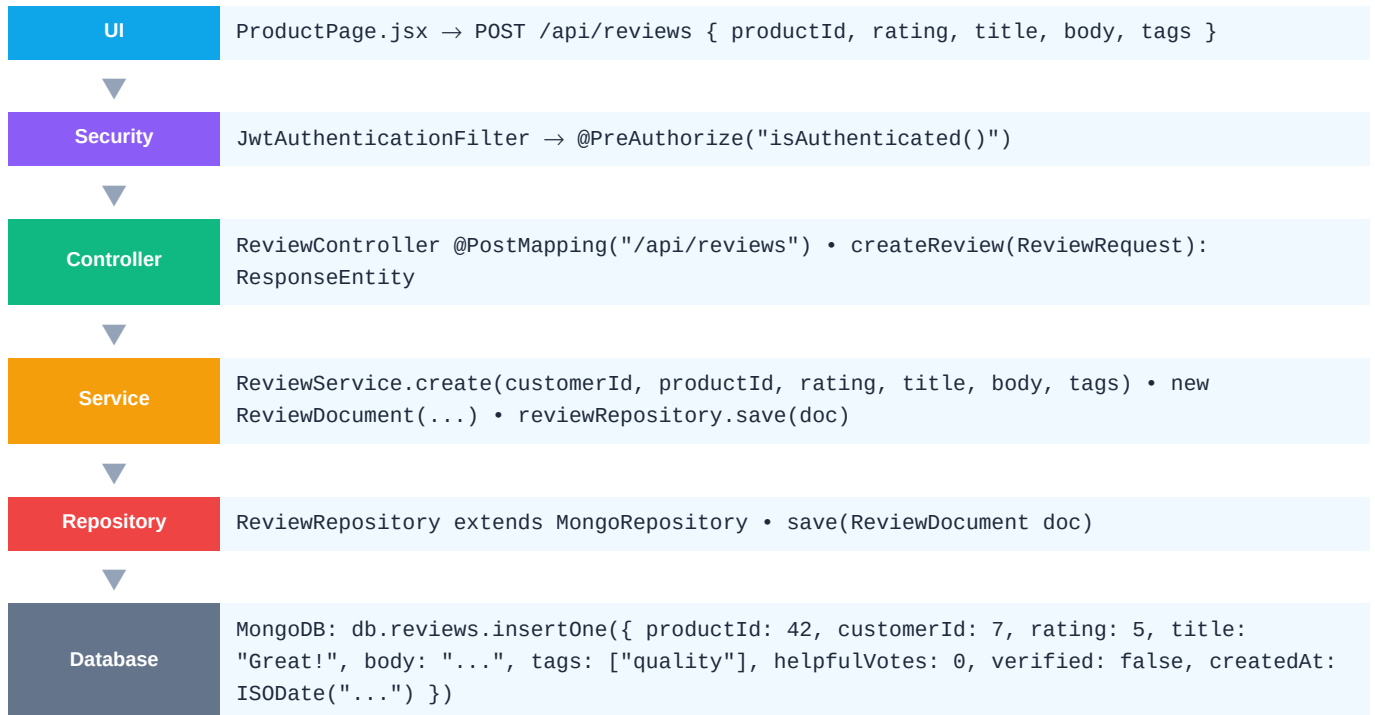
- ✓ Always filter by partition key (`customer_id`) — Cassandra cannot scan without it efficiently
- ✓ The bootstrap `CqlSession` in `CassandraConfig.java` uses 30s timeout — Cassandra starts slowly
- ✓ `spring.cassandra.schema-action=NONE` — table DDL is done by `CassandraConfig`, not Spring Data

MongoDB — Document store — product reviews and rich metadata

Use Case 1: Submit Product Review

Authenticated user submits a review. ReviewDocument stored in MongoDB reviews collection with flexible schema (rating, title, body, tags, verified, helpfulVotes).

Request Flow (UI → DB)



Key Code Details

ReviewRepository.java (shopverse-infrastructure)

```
public interface ReviewRepository extends MongoRepository {
    List findByProductIdOrderByCreatedAtDesc(Long productId);

    @Query("{ 'product_id': ?0, 'rating': { $gte: ?1 } }")
    List findByProductIdAndRatingAtLeast(Long productId, int min);

    @Aggregation(pipeline = {
        "{ $match: { 'product_id': ?0 } }",
        "{ $group: { _id: '$product_id', avg: { $avg: '$rating' }, count: { $sum: 1 } } }"
    })
    RatingStats aggregateRating(Long productId);

    interface RatingStats { Double getAvg(); Long getCount(); }
}
```

ReviewDocument.java (shopverse-infrastructure)

```
@Document(collection = "reviews")
```

```

public class ReviewDocument {

    @Id private String id;

    @Indexed private Long productId;

    @Indexed private Long customerId;

    @Min(1) @Max(5) private int rating;

    private String title;

    private String body;

    private List tags;

    private int helpfulVotes;

    private boolean verified;

    private Instant createdAt;

}

```

Getting Logs

Command	What it shows
<code>docker logs shopverse-mongo --tail 50 -f</code>	Follow live MongoDB logs
<code>docker exec -it shopverse-mongo mongosh --eval "db.setProfilingLevel(1,{slowms:100})"</code>	Enable slow query profiler
<code>docker exec -it shopverse-mongo mongosh shopverse --eval "db.system.profile.find().sort({ts:-1}).limit(5).pretty()"</code>	Last 5 slow queries

Inspecting Values

Command	What it shows
<code>docker exec -it shopverse-mongo mongosh</code>	Open MongoDB shell
<code>use shopverse;</code> <code>db.reviews.find().limit(5).pretty()</code>	First 5 review documents
<code>db.reviews.find({productId:1}).count()</code>	Review count for product 1
<code>db.reviews.aggregate([{\$group:{_id:'\$productId', avg:{\$avg:'\$rating'}}})</code>	Average rating per product
<code>db.reviews.getIndexes()</code>	All indexes on reviews collection

Pro Tips

- ✓ `db.reviews.find({productId:1}).explain('executionStats')` confirms index is used (not COLLSCAN)
- ✓ MongoDB auto-creates the shopverse DB and reviews collection on first Spring Data save()
- ✓ `spring.data.mongodb.uri` defaults to `mongodb://localhost:27017/shopverse` in `application.yml`

Neo4j — Graph database — product recommendations via Cypher

Use Case 1: Product Recommendations (GraphQL productRecommendations)

GraphQL query returns products related to a given product via FREQUENTLY_BOUGHT_TOGETHER relationships in Neo4j, traversed up to 2 hops.

Request Flow (UI → DB)



Key Code Details

ProductGraphQLController.java (shopverse-web)

```
@Controller
public class ProductGraphQLController {
    private final ProductGraphRepository productGraphRepository;
    private final SearchProductsUseCase searchProductsUseCase;

    @QueryMapping
    public List products(
        @Argument String keyword, @Argument String category,
        @Argument Integer page, @Argument Integer size) {
        return searchProductsUseCase.execute(new SearchProductsQuery(...));
    }

    @QueryMapping
    public List productRecommendations(@Argument Long productId) {
```

```

    return productGraphRepository.findRecommendations(productId);
}
}

```

ProductGraphRepository.java (shopverse-infrastructure)

```

public interface ProductGraphRepository extends Neo4jRepository {

    @Query("MATCH (p:Product {productId: $productId})"
+ " -[:FREQUENTLY_BOUGHT_TOGETHER*1..2]-(rec:Product) "
+ "WHERE rec.productId <> $productId "
+ "RETURN DISTINCT rec ORDER BY rec.avgRating DESC LIMIT 10")
    List findRecommendations(Long productId);
}

```

ProductNode.java (shopverse-infrastructure)

```

@Node("Product")

public class ProductNode {

    @Id @GeneratedValue private Long id;

    private Long productId;

    private String name;

    private String category;

    private double avgRating;

    @Relationship(type="FREQUENTLY_BOUGHT_TOGETHER")
    private List frequentlyBoughtWith;

}

```

Getting Logs

Command	What it shows
<code>docker logs shopverse-neo4j --tail 100 -f</code>	Follow live Neo4j logs
<code>docker logs shopverse-neo4j 2>&1 grep ERROR</code>	Errors only
<code>docker exec shopverse-neo4j tail -f /logs/neo4j.log</code>	Application log file
<code>http://localhost:7474 → :sysinfo</code>	Browser UI system metrics

Inspecting Values

Command	What it shows
<code>http://localhost:7474 (user: neo4j, pw: shopverse)</code>	Neo4j Browser visual UI
<code>docker exec -it shopverse-neo4j cypher-shell -u neo4j -p shopverse</code>	Cypher shell
<code>MATCH (n:Product) RETURN n LIMIT 5;</code>	First 5 Product nodes

<pre>MATCH (a:Product)-[r:FREQUENTLY_BOUGHT_TOGETHER] -(b:Product) RETURN a,r,b LIMIT 10;</pre>	View relationship graph
<pre>CALL db.schema.visualization()</pre>	Visual node/edge schema
<pre>CALL db.indexes()</pre>	All indexes

Pro Tips

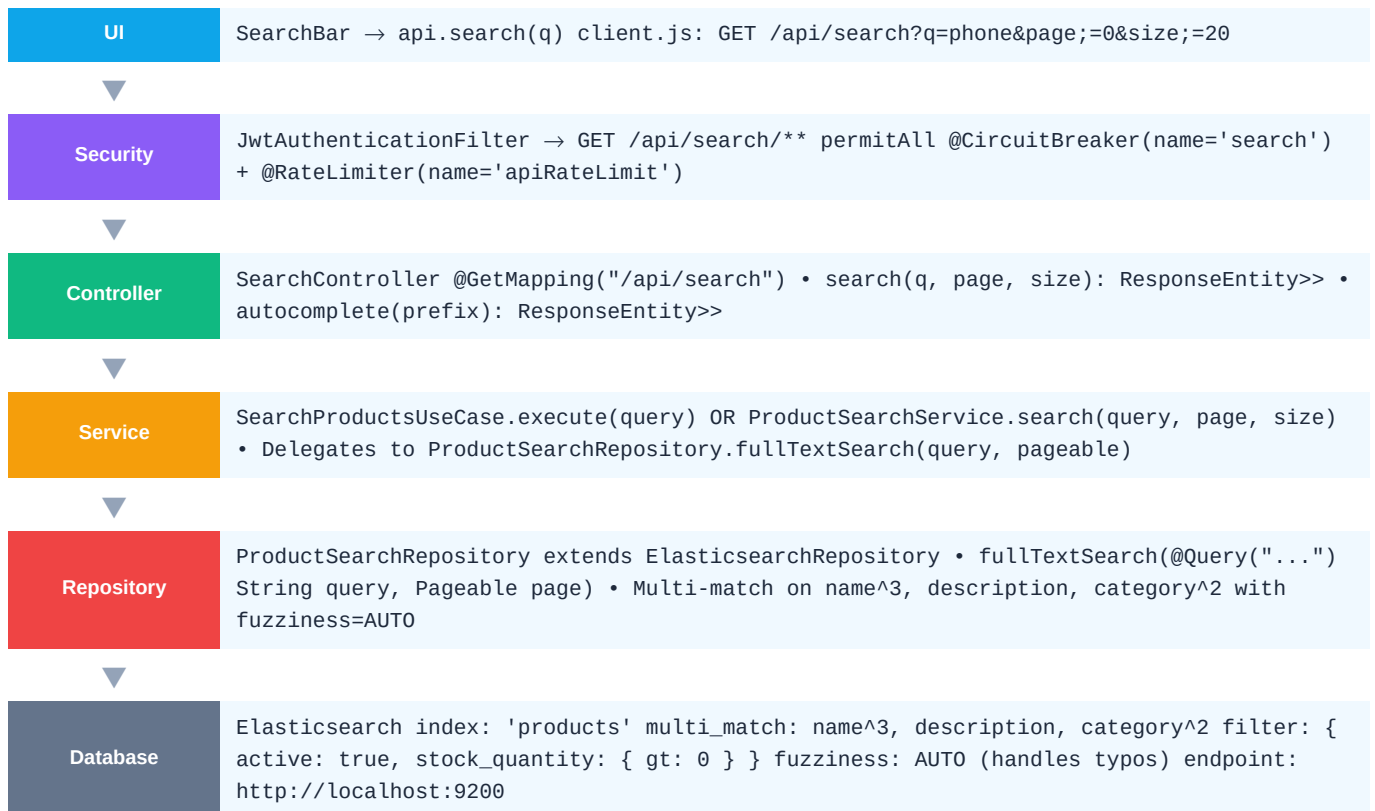
- ✓ Neo4j config: spring.neo4j.authentication.username/password in application.yml (Spring Boot 3.x namespace)
- ✓ Neo4j Browser (port 7474) renders the graph visually — great for understanding data shape
- ✓ PROFILE shows query execution plan and DB hits before optimising

Elasticsearch — Full-text and faceted product search with autocomplete

Use Case 1: Full-Text Product Search (GET /api/search)

User types in search box. SearchController calls SearchProductsUseCase which queries Elasticsearch with a multi_match query, fuzzy matching and active/stock filters.

Request Flow (UI → DB)



Key Code Details

SearchController.java (shopverse-web)

```
@RestController @RequestMapping("/api/search")

public class SearchController {

    @GetMapping

    @CircuitBreaker(name="search", fallbackMethod="searchFallback")

    @RateLimiter(name="apiRateLimit")

    public ResponseEntity>> search(

        @RequestParam String q,

        @RequestParam(defaultValue="0") int page,

        @RequestParam(defaultValue="20") int size) {

        return ok(searchService.search(q, page, size));

    }

    @GetMapping("/autocomplete")
```

```

@CircuitBreaker(name="search", fallbackMethod="autocompleteFallback")

public ResponseEntity>> autocomplete(

@RequestParam String prefix) {

return ok(searchService.autocomplete(prefix));

}

}

```

ProductSearchRepository.java (shopverse-infrastructure)

```

public interface ProductSearchRepository

extends ElasticsearchRepository {

@Query("{\"bool\":{\"must\":[{\"multi_match\":{\"query\":\"?0\",

\"fields\":[\"name^3\",\"description\",\"category^2\"],

\"fuzziness\":\"AUTO\"}}],

\"filter\":[{\"term\":{\"active\":true}},

{\"range\":{\"stock_quantity\":{\"gt\":0}}}]}}}")

Page fullTextSearch(String query, Pageable pageable);

}

```

ProductSyncService.java — keeps ES in sync (shopverse-infrastructure)

```

@Component

public class ProductSyncService {

@Async @EventListener

public void onProductCreated(ProductEvent.ProductCreated event) {

indexProduct(event.product());

}

@Async @EventListener

public void onProductUpdated(ProductEvent.ProductUpdated event) {

indexProduct(event.product()); // upsert into ES

}

private void indexProduct(Product p) {

searchRepo.save(ProductDocument.from(p));

}

}

```

Getting Logs

Command	What it shows
<code>docker logs shopverse-elasticsearch --tail 100 -f</code>	Follow live ES logs
<code>curl -s http://localhost:9200/_cluster/health?pretty</code>	Cluster health: green/yellow/red

<code>curl -s 'http://localhost:9200/_cat/indices?v&s=index'</code>	All indexes with doc count
<code>curl -s http://localhost:9200/_nodes/stats?pretty grep heap</code>	JVM heap per node

Inspecting Values

Command	What it shows
<code>curl -s http://localhost:9200</code>	Version and cluster name
<code>curl -s http://localhost:9200/products/_count</code>	Total docs in products index
<code>curl -s 'http://localhost:9200/products/_search?pretty&size=3'</code>	First 3 documents
<code>curl -s -X POST http://localhost:9200/products/_search?pretty -H 'Content-Type:application/json' -d '{"query":{"match":{"name":"phone"}}}'</code>	Search for 'phone'
<code>curl -s http://localhost:9200/products/_mapping?pretty</code>	Field mappings / index schema
<code>curl -s http://localhost:9200/products/_stats?pretty</code>	Docs, store size, search stats

Pro Tips

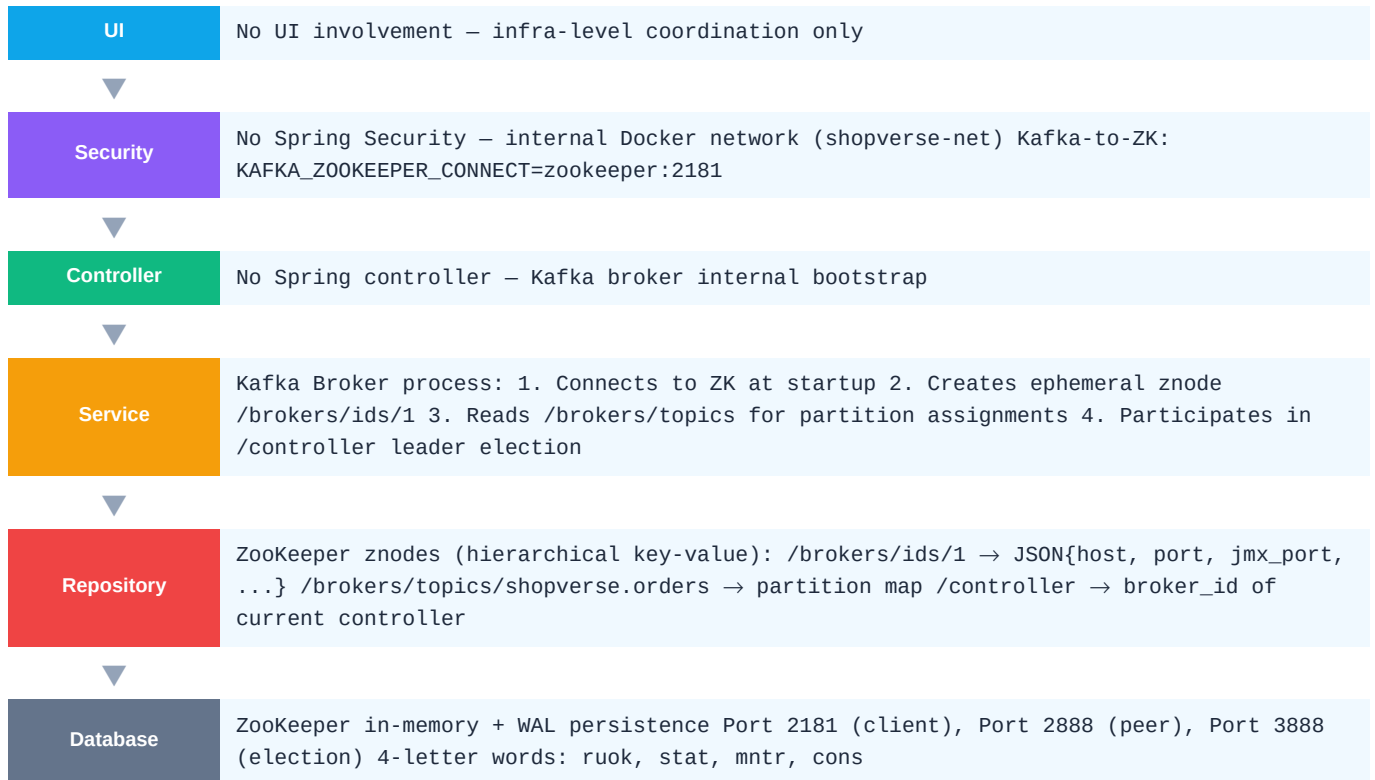
- ✓ Resilience4j @CircuitBreaker on SearchController opens circuit if ES is down — returns fallback empty list
- ✓ ProductSyncService uses @EventListener with @Async — ES sync is non-blocking after product save
- ✓ `curl http://localhost:9200/_cat/health?v` is the quickest ES health check

Zookeeper — Kafka coordinator — broker metadata, leader election

Use Case: Kafka Broker Registration (startup)

When the Kafka container starts it registers itself with ZooKeeper. ZK stores broker metadata, topic-to-partition assignments and elects one broker as the 'controller'.

Request Flow (UI → DB)



Key Code Details

docker-compose.yml — Kafka listener config (shopverse root)

```
kafka:
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT
    KAFKA_LISTENERS: INTERNAL://0.0.0.0:29092,EXTERNAL://0.0.0.0:9092
    KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka:29092,EXTERNAL://localhost:9092
    KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
    # INTERNAL used by Docker services (kafka:29092)
    # EXTERNAL used by host Spring Boot (localhost:9092)
```

Getting Logs

Command	What it shows
---------	---------------

<code>docker logs shopverse-zookeeper --tail 50 -f</code>	Follow live ZK logs
<code>docker exec shopverse-zookeeper bash -c "echo ruok nc localhost 2181"</code>	Health check — returns 'imok'
<code>docker exec shopverse-zookeeper bash -c "echo stat nc localhost 2181"</code>	Stats: latency, connections
<code>docker exec shopverse-zookeeper bash -c "echo mntr nc localhost 2181"</code>	Full metrics dump

Inspecting Values

Command	What it shows
<code>docker exec shopverse-zookeeper bash -c "echo ruok nc localhost 2181"</code>	'imok' = healthy
<code>docker exec shopverse-zookeeper bash -c "echo svr nc localhost 2181"</code>	Leader/follower role
<code>docker exec -it shopverse-zookeeper /bin/zookeeper-shell.sh localhost:2181</code>	ZK shell
<code>ls /brokers/ids</code>	In ZK shell: registered brokers
<code>ls /brokers/topics</code>	In ZK shell: all Kafka topics
<code>get /brokers/ids/1</code>	In ZK shell: broker 1 JSON metadata

Pro Tips

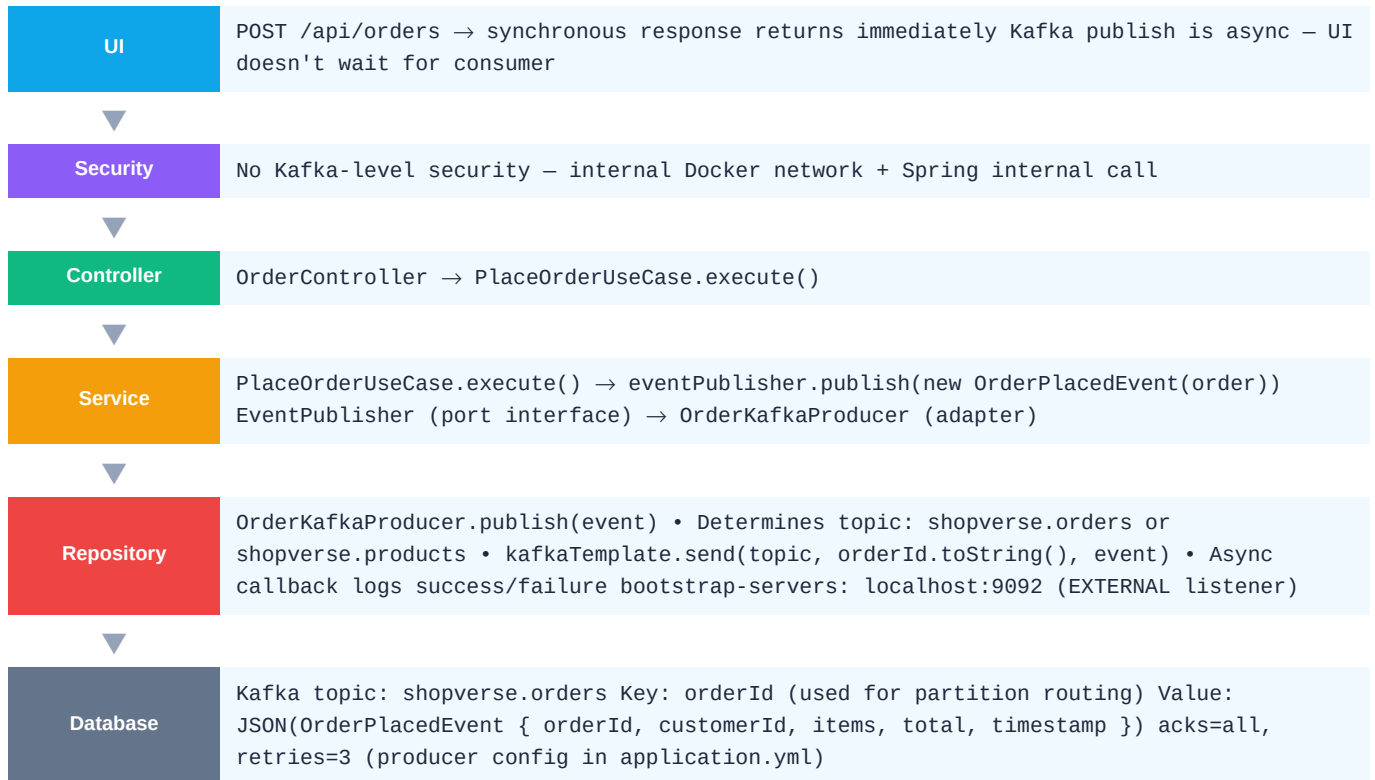
- ✓ `echo ruok | nc localhost 2181` is the fastest ZK health check — 'imok' = all good
- ✓ Kafka 3.x introduces KRaft mode (no ZK) — this app uses legacy ZK mode with Confluent 7.6
- ✓ ZK is only for Kafka coordination — your app code never talks to ZK directly

Kafka — Async event streaming — order events, notifications, DLQ

Use Case 1: Publish Order Placed Event

After PlaceOrderUseCase saves an order to PostgreSQL, it calls eventPublisher.publish(). OrderKafkaProducer sends the event to Kafka topic 'shopverse.orders' asynchronously.

Request Flow (UI → DB)



Key Code Details

OrderKafkaProducer.java (shopverse-infrastructure)

```
@Component

public class OrderKafkaProducer implements EventPublisher {

    private static final String ORDERS_TOPIC = "shopverse.orders";

    private static final String PRODUCTS_TOPIC = "shopverse.products";

    @Override

    public void publish(Object event) {

        String topic = (event instanceof OrderEvent) ? ORDERS_TOPIC : PRODUCTS_TOPIC;

        String key = extractKey(event); // orderId for order events

        kafkaTemplate.send(topic, key, event)

            .whenComplete((result, ex) -> {

                if (ex != null) log.error("Kafka send failed", ex);

                else log.debug("Sent to partition {}, offset {}",

                    result.getRecordMetadata().partition(),
```

```

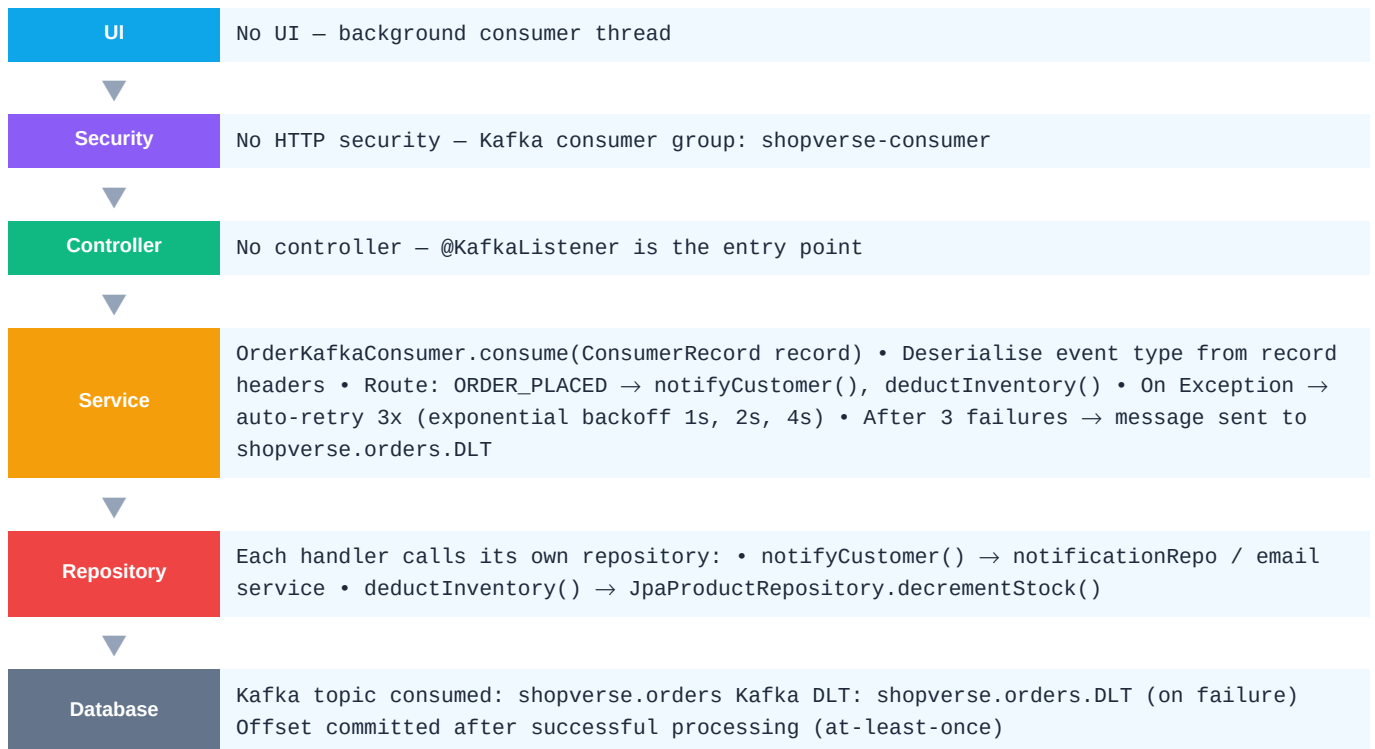
result.getRecordMetadata().offset());
});
}
}

```

Use Case 2: Consume Order Event with Retry and DLQ

OrderKafkaConsumer listens on 'shopverse.orders'. On failure it retries 3 times with exponential backoff. After 3 failures the message lands on shopverse.orders.DLT (Dead Letter Topic) for manual inspection.

Request Flow (UI → DB)



Key Code Details

OrderKafkaConsumer.java (shopverse-infrastructure)

```

@Component
public class OrderKafkaConsumer {

    @RetryableTopic(attempts = "3",
        backoff = @Backoff(delay = 1000, multiplier = 2.0),
        dltTopicSuffix = ".DLT")
    @KafkaListener(topics = "shopverse.orders",
        groupId = "shopverse-consumer")
    public void consume(ConsumerRecord record) {
        Object event = record.value();
        if (event instanceof OrderPlacedEvent e) {
            notifyCustomer(e);
            deductInventory(e);
        }
    }
}

```

```
// Exceptions trigger retry → DLT after 3 attempts
}

@KafkaListener(topics = "shopverse.orders.DLT",
groupId = "shopverse-dlt")
public void consumeDlt(ConsumerRecord record) {
    log.error("DLT message: key={}, value={}",
record.key(), record.value());
    // Alert / manual replay logic here
}
}
```

application.yml — Kafka producer/consumer config (shopverse-web)

```
spring:
  kafka:
    bootstrap-servers: ${KAFKA_BROKERS:localhost:9092}
    admin:
    fail-fast: false # app starts even if Kafka is slow
    producer:
      acks: all # wait for all replicas
      retries: 3
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    consumer:
      group-id: shopverse-consumer
      auto-offset-reset: earliest
      key-deserializer: ...StringDeserializer
      value-deserializer: ...JsonDeserializer
    properties:
      spring.json.trusted.packages: "com.shopverse.*"
```

Getting Logs

Command	What it shows
<code>docker logs shopverse-kafka --tail 100 -f</code>	Follow live Kafka logs
<code>docker logs shopverse-kafka 2>&1 grep ERROR</code>	Error lines
In application.yml: <code>org.springframework.kafka: DEBUG</code>	Spring Kafka producer/consumer debug
<code>docker exec shopverse-kafka kafka-consumer-groups --bootstrap-server localhost:9092 --describe --group shopverse-consumer</code>	Consumer LAG per partition

Inspecting Values

Command	What it shows
<code>docker exec shopverse-kafka kafka-topics --bootstrap-server localhost:9092 --list</code>	List all topics
<code>docker exec -it shopverse-kafka kafka-console-consumer --bootstrap-server localhost:9092 --topic shopverse.orders --from-beginning</code>	Read all order events
<code>docker exec -it shopverse-kafka kafka-console-consumer --bootstrap-server localhost:9092 --topic shopverse.orders.DLT</code>	Watch dead letter topic
<code>docker exec -it shopverse-kafka kafka-console-producer --bootstrap-server localhost:9092 --topic shopverse.orders</code>	Publish test message
<code>docker exec shopverse-kafka kafka-get-offsets --bootstrap-server localhost:9092 --topic shopverse.orders</code>	Latest offset per partition

Pro Tips

- ✓ Consumer LAG = latest_offset - committed_offset — high lag means consumers can't keep up
- ✓ `kafka-console-consumer --from-beginning` replays all events — great for testing consumer logic
- ✓ Kafka EXTERNAL listener (localhost:9092) is for host Spring Boot; INTERNAL (kafka:29092) is Docker-to-Docker